R3.01 développement web

R3.01 développement web

Date de publication 2026

Table des matières

1. Présentation 1
    1. Objectifs 1
    2. Environnement de travail 1
        2.1. Utilisation d'un conteneur léger pour héberger notre serveur 1
        2.2. Un premier code java 2
            2.2.1. Structure d'un projet JEE 2
            2.2.2. Nos premier codes 2
            2.2.3. La compilation et le déploiement 3
2. JEE (servlet, JSP, JPA) 6
    1. Apprentissage de java 6
        1.1. Différences entre java et Kotlin 6
        1.2. Un premier code en java 6
            1.2.1. Un tutoriel 6
            1.2.2. La promotion : Classe et héritage 7
            1.2.3. Mise en oeuvre du design pattern DAO 10
    2. Les servlet 12
        2.1. Cycle de vie 12
        2.2. Les annotations @WebServlet/@WebInitParam 12
        2.3. Un formulaire 13
        2.4. Une redirection 13
        2.5. Les cookies et les sessions 14
            2.5.1. La personnalisation 14
            2.5.2. La gestion des sessions 15
    3. JSP 15
        3.1. Les bases 16
            3.1.1. HelloWorld 17
            3.1.2. Afficher un tableau de produit 17
            3.1.3. Afficher un tableau de produits depuis un dao via une servlet 17
            3.1.4. Augmenter et diminuer la quantité de produits 17
            3.1.5. Login 18
    4. Java Persistence API 18
        4.1. La structure du projet 18
        4.2. Une première découverte 18
            4.2.1. L'entité produit 18
            4.2.2. Utilisation d'une session 19
            4.2.3. Utilisation d'un gestionnaire d'entités 21
        4.3. Un Dépôt simple 22
        4.4. Associations OneToMany 22
        4.5. Associations ManyToMany 24
        4.6. ManyToMany avec des paramètres 25
3. Une première application MVC 27
4. Utilisation du framework MVC spring boot 28

Liste des illustrations

2.1. Classe Etudiant 7
2.2. EtatMental 8
2.3. EtudiantAvecEtatMental 9
2.4. Etudiant avec comptage 9
2.5. Promotion 10
2.6. Les cookies HTTP 14
2.7. Cycle de vie JSP. 16

Chapitre 1. Présentation

1. Objectifs

Dans la ressource R1.02:Développementd'interfaces web, vous avez appris à réaliser des pages web statiques. Dans cette ressource nous allons produire des pages Web dynamiques via un serveur HTTP (Hypertext Transfer Protocol). Le langage choisi est Java avec Jakarta Enterprise Edition (JEE), un langage de script impératif orienté objet. Pour la persistance de nos données nous utiliserons le SGBD (Système de Gestion de Base de Données) H2.

Nous disposons dans cette ressource de 5 créneaux de 1h20 de CM et de 22 créneaux de TD. Voici une progression indicative :

Semaines	CM	TD	TD de SAE	Évaluation	Objectifs
S40	1h20	2h40			Apprentissage JAVA
S41	1h20	4h			Apprentissage JAVA + Servlet
S42	1h20	4h			Servlet / JSP
S43	1h20	4h			JSP
S45	1h20	4h			JPA
S46				2h40	JPA
S47		4h			MVC
S48		2h40	1h20		MVC
S49			4h		Spring Boot
S50			1h20	2h40	Spring Boot
S51			?		Spring Boot
S1			?		Soutenance SAE

2. Environnement de travail

Nous allons créer et exécuter notre première servlet. Une servlet est une classe Java qui permet de créer dynamiquement des données au sein d'un serveur HTTP. Il nous faut donc un serveur HTTP et un moteur de servlet (conteneur web) qui prend en charge et gère les servlets. Nous allons utiliser tomcat (`https://tomcat.apache.org/`) un moteur léger.

2.1. Utilisation d'un conteneur léger pour héberger notre serveur

Lors du S4 vous aurez des cours de virtualisation qui vous permettrons de réaliser les scripts présentés ici. Nous allons podman, un logiciel libre compatible avec docker permettant de lancer des applications dans des conteneurs logiciels.

Étant derrière un proxy, pour ne pas créer de déni de service (DoS), il vous faut créer un compte sur `https://www.docker.com/`. Ce compte sera utilisé pour le téléchargement des images.

Vous disposez des scripts suivants :

docker-compose.yml

> script qui permet la création du conteneur et la définition du point de montage, le ./tomcat/webapps sera monté dans /usr/local/tomcat/webapps du conteneur.

create.sh

script qui vous demande de vous authentifier et qui lance en arrière plan les services définis dans docker-compose.yml

down.sh

script qui stop les services avant de vous déloguer

terminal.sh

script qui permet d'obtenir un terminal dans le conteneur

Mettre les droits d'exécution sur les scripts bash (**chmod u+x *.sh**) puis tester l'URL `http://localhost:8888/`, vous devriez obtenir un code 404, le serveur répond mais n'a pas de données à fournir.

2.2. Un premier code java

Notre premier programme est un projet JEE est doit respecter une structure :

2.2.1. Structure d'un projet JEE

```
src
   ### main
   #   ### java
   #   #   ### iut
   #   #       ### nantes
   #   #           ### demo
   #   #               ### HelloServlet.java
   #   ### resources
   #   ### webapp
   #       ### WEB-INF
   #       #   ### web.xml
   #       ### index.jsp
   ### test
       ### java
       ### resources
```

src/main/java

Contient tout le code source Java

src/main/ressources

Contient les fichiers de configuration

src/main/webapp/

est la racine du contenu web : HTML, CSS, JS, JSP (Jakarta Server Pages), ...

src/main/webapp/WEB-INF

est un dossier protégé contenant le descripteur de déploiement (web.xml) et les pages sécurisées.

src/test/java

est le dossier contenant le code source des tests unitaires et d'intégration

2.2.2. Nos premier codes

2.2.2.1. Une servlet

Le constituant de base est la servlet, une classe Java qui permet de créer dynamiquement des données au sein d'un serveur HTTP. Les servlet disposent d'un cycle de vie et réponde à des requêtes HTTP.

Dans l'exemple suivant si la route de l'application /hello-servlet est sollicitée avec le verbe HTTP GET, alors la méthode doGet est invoquée avec deux paramètre, la requête et la réponse.

```
package iut.nantes.demo;

import java.io.*;

import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;

@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {
    private String message;

    public void init() {
        message = "Hello World!";
    }

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws IOException {
        response.setContentType("text/html");

        // Hello
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h1>" + message + "</h1>");
        out.println("</body></html>");
    }

    public void destroy() {
    }
}
```

2.2.2.2. Un JSP

Un JSP est une technique qui permet à partir d'un langage XML de créer des servelts. Ici nous aurons un titre et un lien vers la servlet précédente.

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html>
<head>
    <title>JSP - Hello World</title>
</head>
<body>
<h1><%= "Hello World!" %>
</h1>
<br/>
<a href="hello-servlet">Hello Servlet</a>
</body>
</html>
```

2.2.3. La compilation et le déploiement

Nous avons du code, il nous faut le compiler pour obtenir du byte code, des .class et déployer ce byte code sur le serveur.

2.2.3.1. Le moteur de production

Il nous faut un moyen d'automatiser la production du code exécutable (compilation et assemblage), avec une gestion et une détection des dépendances, nous allons utiliser gradle mais vous trouverez également des solutions avec maven ou encore ant.

Structure d'un projet gradle :

```
gradle-project
### app
#   ### build.gradle
```

```
#   ### ...
### settings.gradle
### gradle
#   ### ...
### build
### gradlew
### gradlew.bat
```

gradlew

un script qui télécharge et exécute automatiquement la version de Gradle appropriée pour un projet.

build

la configuration au niveau du projet

setting.gradle

pour intégrer le projet et construire une identité

gradle

Exécutables (wrapper) » et version

Le fichier qui nous intéresse est build.gradle, nous y trouvons à minima :

l'identifiant du groupe et la version

```
group 'iut.nantes'
version '1.0-SNAPSHOT'
```

des plugins

```
plugins {
    id 'war'
}
```

Nous aurons besoin de war pour le déploiement

de dépôts

```
repositories {
    mavenCentral()
}
```

de dépendances

```
dependencies {
    compileOnly('jakarta.servlet:jakarta.servlet-api:6.2.0-M2')
}
```

des versions

```
java {
    sourceCompatibility = JavaVersion.VERSION_21
    targetCompatibility = JavaVersion.VERSION_21
}[1]
```

Gradle repose sur la notion de tache, une action atomique pour une compilation. Vous pouvez obtenir la liste des tâches avec **./gradlew tasks**.

2.2.3.2. Les fichiers de déploiement

Pour déployer du code sur un serveur tomcat, il faut déposer déposer le code compilé, avec les descripteurs, les éventuelles ressources et librairies dans un le dossier ou un sous dossier du répertoire webapps du serveur.

[1]Cette version est celle installée dans les salles, nous pouvons utiliser une version plus récente en passant par notre IDE.

Le serveur tomcat dont nous disposons est configuré pour déployer sans redémarrage à chaque dépôt dans /usr/local/tomcat/webapps que nous avons monté dans répertoire local ./tomcat/webapps.

A vous trouver la tache gradle pour générer le war, puis de déposer le war créé dans le répertoire webapps de tomcat, le déploiement devrait-être automatique. Vous, pouvez tester avec l'URL du la page JSP, le lien conduisant à la servlet : http://localhost:8080/demo-1.0-SNAPSHOT/index.jsp*Le localhost n'étant pas toujours disponible à l'IUT vous devrez le remplacer par votre adresse IP 172.16.XX.XX obtenue avec la commande* ***ip a***.

demo-1.0-SNAPSHOT est la concaténation de rootProject.name de settings.gradle et de version de build.gradle, il est possible de choisir le nom dans build.gradle en utilisant la propriété archiveFileName de war, à vous de tester :

```
war {
    archiveFileName = 'demo.war'
}
```

Si vous modifiez votre code, il vous faut le redéployer en supprimant le war déposé, puis en le remplaçant, nous pouvons automatiser tout cela avec des taches.

```
tasks.register('deleteMyWar',Delete) {

    doLast {
        println("Suppression du war : ./tomcat/webapps/${war.archiveFileName.get()}")
        println("Done")
    }
    delete files("./tomcat/webapps/${war.archiveFileName.get()}")

}
```

```
tasks.register('copyMyWar',Copy){

    doLast {
        println("Copy ${war.archiveFileName.get()}
          from ${project.buildDir}/libs to ./tomcat/webapps")
        println("Done")
    }
    from "${project.buildDir}/libs"
    into  "./tomcat/webapps"
    include "${war.archiveFileName.get()}"
}
```

Il ne reste plus qu'à supprimer, le war, le recréer et le copier en faisant attention que la suppression soit finie.

Chapitre 2. JEE (servlet, JSP, JPA)

Nous allons commencer par un tutori

1. Apprentissage de java

1.1. Différences entre java et Kotlin

Java est plus ancien que Kotlin avec une syntaxe plus verbeuse mais un meilleur support. Java est comme Kotlin un langage objet, fortement typé et exécuté sur une JVM. Voici avant quelques différences :

null safety

: Java autorise l'affectation de valeurs nulles aux objets. Par conséquent, l'accès à un membre d'une référence nulle provoque à l'exécution une exception : NullPointerException.

Extension Functions

: Les fonctions d'extension n'existent pas en java, le seul moyen pour étendre les méthodes est d'hériter.

Typage strict vs. typage statique

: Java est un langage à typage strict, ce qui signifie que toutes les variables sont déclarées avec un type lors de leur création. En revanche, Kotlin est un langage à typage statique qui peut déterminer le type d'une variable en fonction du type de la valeur qui lui est assignée (inférence de type).

Casting

: En Kotlin le compilateur peut générer automatiquement les transtypages, en java ils devront être explicites.

Programmation fonctionnelle

: Kotlin est à la fois orienté objet et fonctionnel, ce qui n'était pas le cas de java pour les premières versions de java. Les lambda existent pas les fonctions d'extensions et les fonctions déléguées.

Coroutines

: Java dispose des threads mais pas des coroutines

Classes scellées

: En java, les classes sont ouvertes par défaut, si vous souhaitez limiter l'héritage, il faudra utiliser les mots clefs final ou sealed.

1.2. Un premier code en java

Vous pouvez utiliser l'IDE de votre choix et créer un projet java classique, nous n'aurons pas besoin de JEE pour le moment. Un projet gradle avec des tests est fourni.

1.2.1. Un tutoriel

Je vous propose de commencer par un tutoriel officiel java portant sur les classes et les objets : `https://docs.oracle.com/javase/tutorial/java/javaOO/index.html`. Vous pouvez parcourir rapidement la partie cours mais n'oubliez pas la partie exercices. Vous devrez aller jusqu'aux énumérations. Par rapport à Kotlin, vous devriez apprendre qu'une classe est ouverte par défaut, que l'héritage s'exprime avec le mot clef `extends`, que la

réalisation d'une interface est exprimée avec `implements` que le chaînage des constructeurs s'exprime avec `super()`, que les constructeurs peuvent-être surchargés et appelé via `this()` que les objets sont instanciés avec `new`. Les "compagnions" peuvent être assimilés aux attributs statique et aux classes statiques imbriquées mais avec sans accès directe, il faut passer par le nom de la classe.

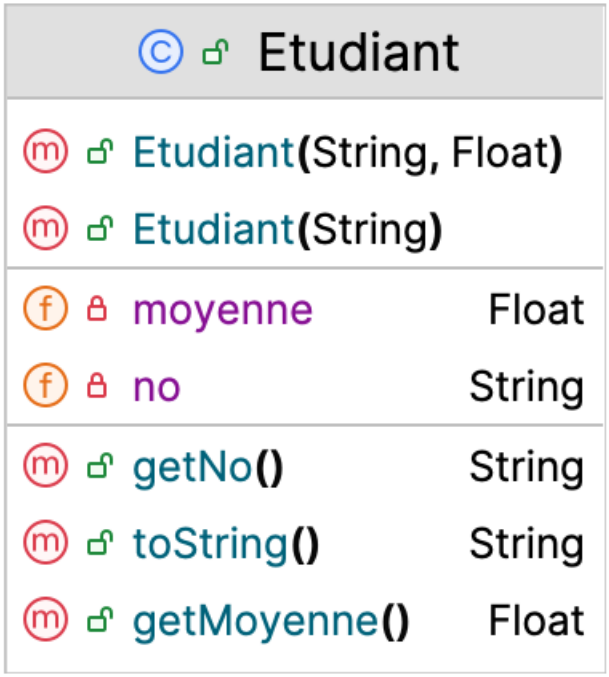
1.2.2. La promotion : Classe et héritage

Nous allons coder une Promotion composée d'étudiants, à chaque étape, vous devrez passer les tests.

1.2.2.1. Un étudiant

Créer une classe Etudiant avec deux attributs privés no une string et une moyenne un flottant, les méthode getNo() et getMoyenne() permettront de consulter les attributs, un constructeur permettra de construire un étudiant avec un no et une moyenne choisis, si la moyenne n'est pas fournie, elle sera supposée égale à 10.

Figure 2.1. Classe Etudiant



Créer un classe avec un main pour créer deux étudiants en utilisant les deux constructeurs et afficher les propriétés.

Dans ce formalisme :

f

: Représente un champ ou un attribut de la classe

m

: Représente une méthode ou un constructeur

Les losanges

: Les propriétés où les méthodes de classe

C

: Représente un classe

I

Représente une interface

E

Une énumération

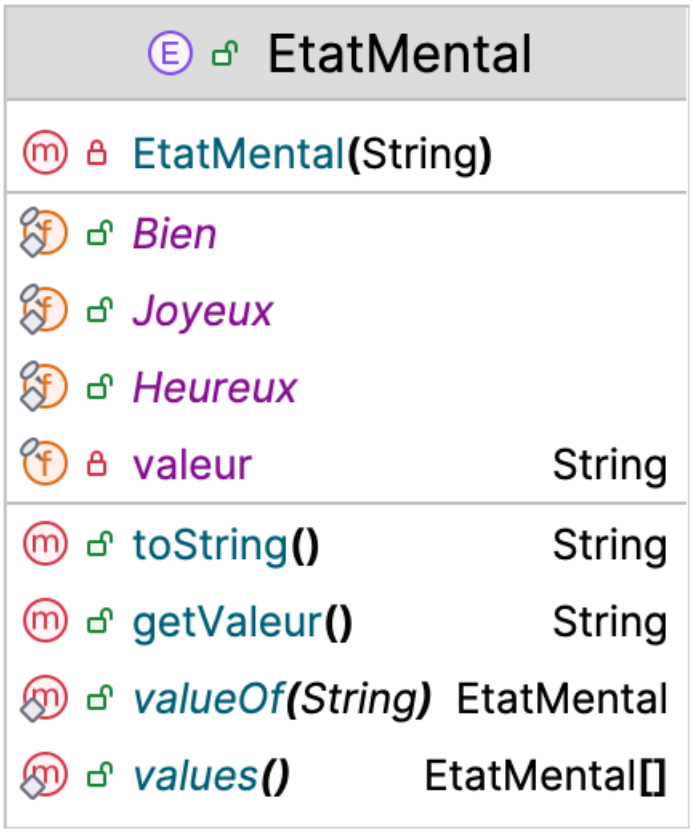
Les cercles en pointillés

Représentent des classes ou des méthodes abstraites

1.2.2.2. Une énumération

Vous trouvez ici la document java d'une énumération : `https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html`. Cette énumération dispose d'un constructeur et d'un attribut valeur de type string. Les méthode values() et valueOf() sont natives et ne sont pas à coder.

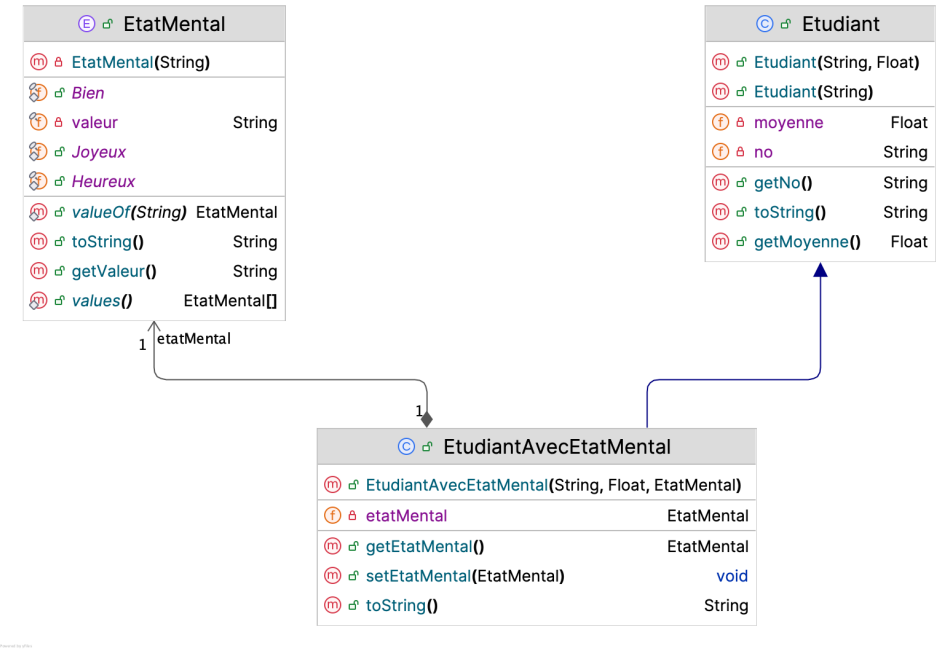
Figure 2.2. EtatMental



1.2.2.3. Un étudiant avec un état mental

Créer une classe fille EtatMental qui représente un Etudiant avec un EtatMental.

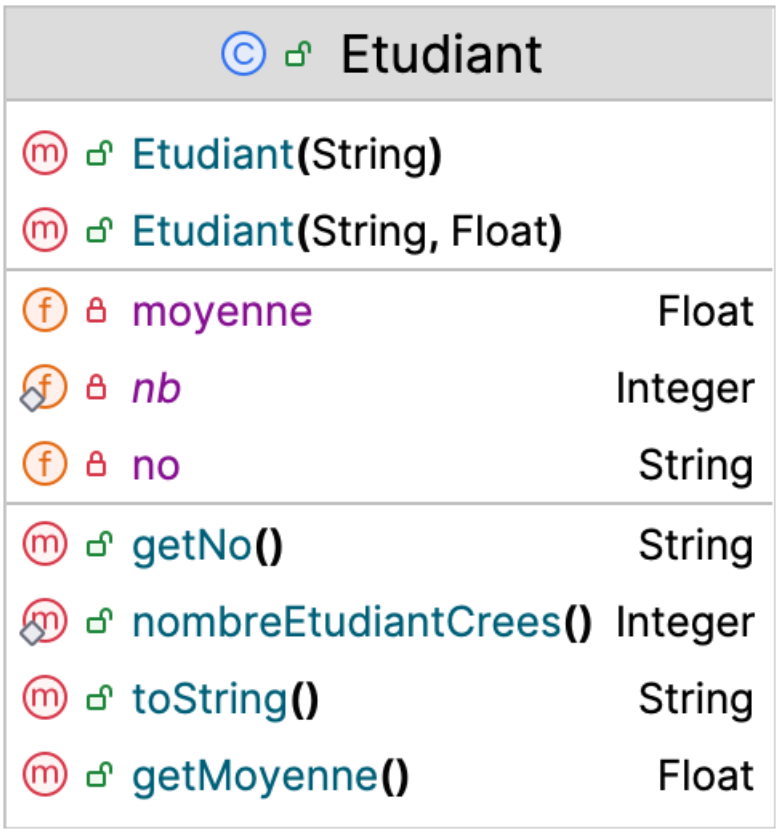
Figure 2.3. EtudiantAvecEtatMental



1.2.2.4. Connaître le nombre de tous les étudiants créer

En utilisant un attribut et une méthode static pour comptabiliser et récupérer les étudiants créés.

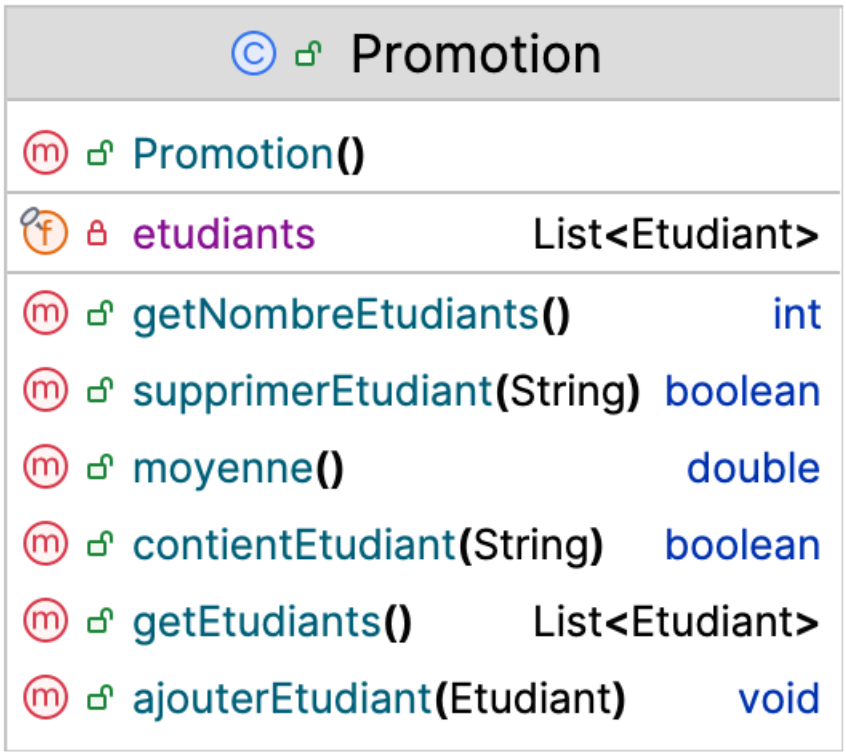
Figure 2.4. Etudiant avec comptage



1.2.2.5. La promotion

Implémenter et tester une promotion d'étudiants, un étudiant caractérisé par son numéro ne pourra être ajouté deux fois cela devra déclencher une Exception. La suppression d'un étudiant non membre renvoiera quand à elle false. Vous devrez également fournir la moyenne de la promotion, si la promotion est vide alors la moyenne sera nulle.

Figure 2.5. Promotion



1.2.3. Mise en oeuvre du design pattern DAO

Nous allons mettre en oeuvre le design pattern DAO, de manière classique pour en utilisant les Optionals, une solution proposée par java pour répondre au problème de sûreté des nulls.

1.2.3.1. Approche classique

Le travail est à réaliser dans le paquage old, l'interface DAO, la classe Product ainsique les jeux de tests sont fournis, vous devez coder la classe ProductMemoryDao qui implémente un DAO pour des products. Je vous conseil d'utiliser pour son implémentation une HashMap (`https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html`).

1.2.3.2. Approche Optional

Les "Optional" ont été créés en java 8 pour répondre au problème des nulls pointeur exception (`https://www.oracle.com/technical-resources/articles/java/java8-optional.html`).

Les "Optional" encapsule un type pour forcer la gestion des nulls. Il constituent avec les stream (`https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html`) un moyen d'avoir une approche fonctionnelle sans null pointeur exception. Il est à noter que les "Optional" ne sont pas sérialisables.

Voici un mini guide de survie :

Une optional vide

```
Optional<String> myOptional = Optional.empty();
```

Une option sur un élément non null

```
Optional<String> myOptional = Optional.of(element);
```

Une optional sur un élément potentiellement null

```
Optional<String> myOptional = Optional.ofNullable(element);
```

Cette méthode permet également de créer une optional qui encapsule null

```
Optional<String> myOptional = Optional.ofNullable(null);
```

Présence d'un élément

```
myOptional.isEmpty()
```

ou

```
myOptional.isPresent()
```

Récupérer un élément avec possibilité de NoSuchElementException

```
myOptional.get()
```

Récupérer un élément avec une valeur par défaut

```
myOptional.orElse(ValeurParDefaut)
```

Récupération avec une valeur par défaut issue d'un Supplier

Un supplier est souvent utilisé sous sa forme lambda (`https://docs.oracle.com/javase/8/docs/api/java/util/function/Supplier.html`).

Cette approche permet de renvoyer une valeur par défaut issue d'un supplier si l'optional est vide.

```
myOptional.orElseGet(() -> ValeurParDefaut))
```

Cette approche est plus performante que la précédente en ce sens que la lambda n'est exécutée que si l'optional n'est pas vide.

Lever une exception la valeur est absente

```
Optional.ofNullable(element).orElseThrow();
```

ou

```
Optional.ofNullable(element)
                    .orElseThrow(() -> new Exception(""));
```

Les filtres

```
myOptional.filter(lambda)
```

Les filtres répondent au design pattern bulder

```
myOptional.filter(lambda).filter(lambda).isPresent()
```

Les mappings

```
myOptional.map(Methode).orElse(Valeur)
```

Implémenter dans le package Optional l'interface DAO qui a été modifiée avec :

```
Optional<T> get(long id);
```

Créer également des jeux de tests.

Cette nouvelle est plus adaptée à l'utilisation de la Java Persistance API que nous utiliserons plus tard et dans laquelle la méthode find renvoie des Optionals.

2. Les servlet

2.1. Cycle de vie

Une servlet sera pour nous une classe fille de HttpServlet qui s'exécute sur un serveur web pour recevoir des requêtes HTTP (Get, Post) et fournir une réponse HTML.

Pour exécuter un servlet, il faut utiliser une requête HTTP, si le servlet est associé à l'URL alors si c'est la première sollicitation, le serveur l'instancie. La servlet reste en mémoire jusqu'à l'arrêt du serveur. La servlet peut être appelée par plusieurs threads lancés par le serveur.

Le cycle de vie d'une servlet Java est géré par le conteneur web (comme Tomcat). Il comprend trois phases principales : l'initialisation via init(), le traitement des requêtes via service() (qui aiguille vers doGet() ou doPost()), et la destruction via destroy(), de sa création à sa suppression.

Créer un package cycle et créer dans ce package une servelet nommée CycleServlet associé au chemin d'URL / cycle-servlet, dans cette servlet redéfinir le constructeur, les méthodes init(),init(ServletConfig config), doGet, doPost, doPut, doPatch, doDelete, destroy dans chaque méthode . Déployer la servlet et la tester, vous pouvez utiliser les commandes curl :

```
curl --noproxy "*" -X GET "http://localhost:8888/demo/cycle-servlet
curl --noproxy "*" -X POST "http://localhost:8888/demo/cycle-servlet
curl --noproxy "*" -X PUT "http://localhost:8888/demo/cycle-servlet
curl --noproxy "*" -X PATCH "http://localhost:8888/demo/cycle-servlet
curl --noproxy "*" -X DELETE "http://localhost:8888/demo/cycle-servlet
```

Pour accèder au log de tomcat, vous pouvez utiliser terminal.sh et accède au log avec **tail -f logs/nom_du_log**. Pour accèder aux logs du terminal, il vous faudra utiliser **podman logs tomcat_container**.

Si vous utilisez super.doUnVerbeHTTP(), comme super.doGet(), il renvoie par défaut un code 405, vous pouvez l'ignorer. De même, vous ne pourrez observer le destroy().

2.2. Les annotations @WebServlet/@WebInitParam

L'objectif de cette annotation est d'associer une classe Java à un ou plusieurs motifs d'URL (urlPatterns ou value) pour traiter les requêtes HTTP. La documentation complette est ici : `https://jakarta.ee/specifications/servlet/4.0/apidocs/javax/servlet/annotation/webservlet`.

Les attributs qui nous intéressent pour le moment sont :

name

: Nom de la servlet (par défaut, le nom complet de la classe)

description

: La description

urlPatterns

Tableau des chemins d'accès déclenchant la servlet, il faut à minima un chemin

initParams

Les paramètre d'initialisation de la servlet (sans rapport avec urlPatterns)

Par exemple vous pourriez avoir :

```
@WebServlet(
  name = "BankAccountServlet",
  description = "Represents a Bank Account and it's transactions",
  urlPatterns = {"/account", "/bankAccount" },
  initParams = {
      @WebInitParam(name = "type", value = "savings"),
      @WebInitParam(name = "name", value = "a name")
})
```

Des wilcards peuvent également être utilisés dans les urlPatterns.

Créer une servlet TestWebServlet qui pour les chemins d'URL /webservlet1 et /webservlet2 accedés en Get affichera dans la console les paramètres, et qui pour valeur et un pas reçus à l'initialisation, affichera les paramètres et la valeur augmentée du pas à chaque consultation en get.

```
curl --noproxy "*" -X GET "http://localhost:8888/demo/webservlet1
#-----------doGet-----------
#valeur : 2
curl --noproxy "*" -X GET "http://localhost:8888/demo/webservlet2
#-----------doGet-----------
#valeur : 4
#curl --noproxy "*" -X GET "http://localhost:8888/demo/webservlet1?nom=Joe&age=18"
-----------doGet-----------
#nom = [Joe]
#age = [18]
#valeur : 6
curl --noproxy "*" -X GET "http://localhost:8888/demo/webservlet1?nom=Joe&age=18&age=22"
#-----------doGet-----------
#nom = [Joe]
#age = [18, 22]
#valeur : 8
```

Les méthodes suivantes peuvent vous aider :

- public abstract java.util.Map<String, String[]> getParameterMap() de ServletRequest
- public String getInitParameter(String name) de GenericServlet

2.3. Un formulaire

Le but est ici de renvoyer une page web qui contient les données d'un formulaire HTML contenant un login et un mot de passe, que les données soient reçue en Get ou en Post. Pour rappel le formulaire HTML, doit-être dans le répertoire webapp. Pour tester, vous pouvez utiliser votre formulaire ou des curl :

```
curl -i -X POST \
  -d "login=admin" \
  -d "password=secret" \
  http://localhost:8080/ton-projet/ta-servlet
```

2.4. Une redirection

Une redirection HTTP fonctionne comme suit :

1. Le client demande une ressource au serveur ;

2. le serveur répond au client avec un code 30X que la ressource est disponible à une autre URL et lui fourni l'adresse;

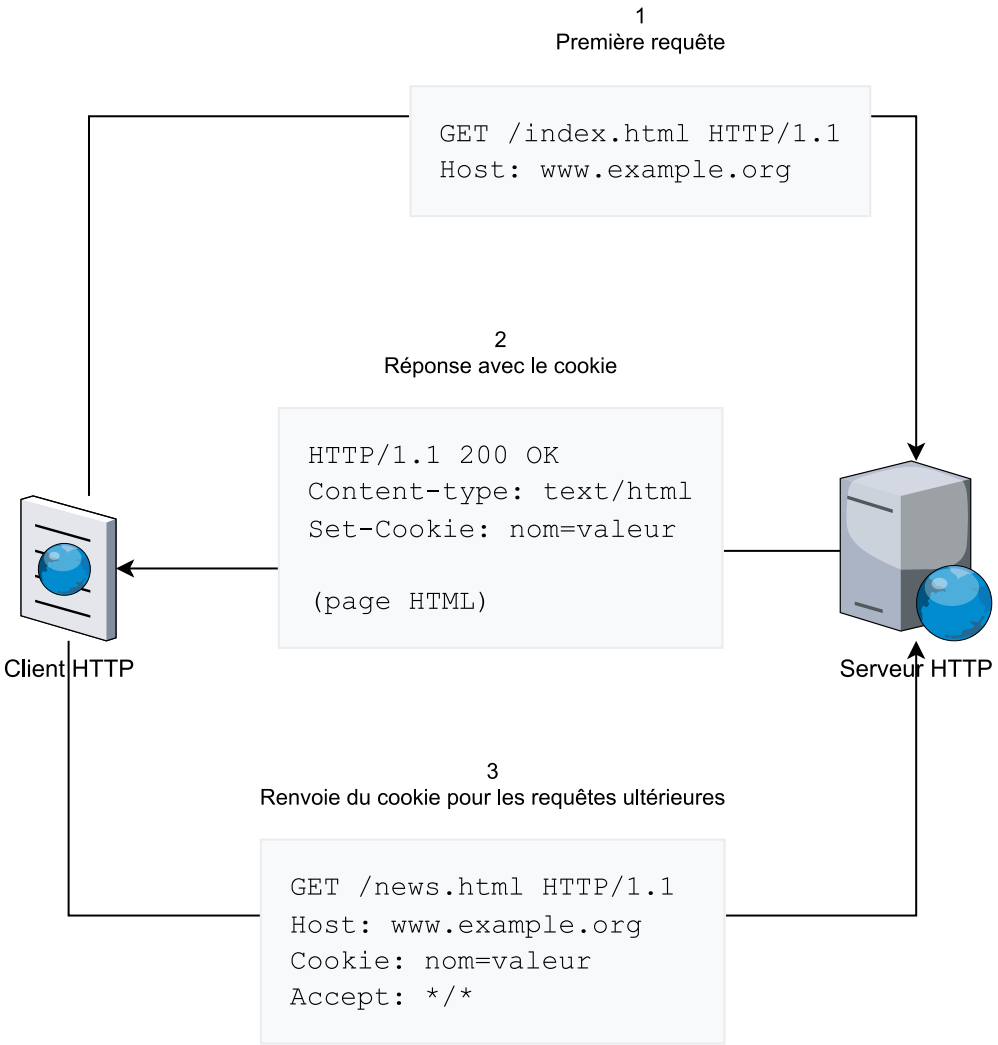
3. le client consulte la nouvelle URL.

En reprenant, l'exemple précédant faite en sorte que si le login n'est pas "Joe" ou le password n'est pas JoJo44 alors le client est redirigé vers le formulaire d'origine. La redirection se fait avec la méthode sendRedirect de HttpServletResponse et l'origine peut-être extraite du champ Referer du header avec la méthode getHeader de HttpServletRequest.

2.5. Les cookies et les sessions

Le protocole HTTP est protocole sans état qui n'enregistre pas l'état de session lors d'une communication entre deux requêtes successives. Cependant le protocole HTTP permet de remédier à ce problème en utilisant des cookies. Un cookie est texte court envoyé par un serveur HTTP à un client HTTP, que ce dernier renvoiera les prochaines fois qu'il se connectera aux serveurs partageant le même nom de domaine.

Figure 2.6. Les cookies HTTP



Les cookies sont utilisés pour la personnalisation, la gestion des sessions, le pistage, ...

2.5.1. La personnalisation

Depuis HTML5 les données peuvent-être validées par le client et la mémoire conservée dans le navigateur, les mêmes fonctionnalités peuvent être offertes par JEE. Nous allons créer une servlet ()qui produit un formulaire

persistant qui demande le login et le mot de passe, si le login et le password sont valides et si ce formulaire est consulté à nouveau alors le login doit rester pré-rempli, ce formulaire aura pour cible une servlet qui créera le cookie.

Si vous testez depuis votre navigateur, vous pouvez supprimer les cookies depuis votre navigateur (menu contextuel puis inspecter).

Vous devrez utiliser la classe jakarta.servlet.http.Cookie , dans le formulaire récupérer les cookies dans la requête et positionner un cookie dans la réponse de la servlet d'authentification.

2.5.2. La gestion des sessions

Nous allons maintenant si l'authentification réussie rediriger vers une servlet qui affichera le login. Vous aurez besoin de la classe jakarta.servlet.http.HttpSession et des méthodes setAttribute et getAttribute.

3. JSP

Dans nos servlet nous avons combiné l'affichage avec les données et les actions, à terme il faudrait que nos sevrlets deviennent des contrôleurs, les Jakarta Server Page deviendront les vues. JSP est une technologie Java qui permet de créer des pages web dynamiques en mélangeant du code HTML (ou XML) et du code Java. Une page JSP est transformée en servlet avec le cycle de vie suivant :

traduction (transpilation)

: A la première consultation de la page, le conteneur analyse le fichier .jsp et le transforme en servlet mon.jsp devient mon_jsp.java

compilation

: Le compilateur Java transforme le fichier .java en bytecode (.class).

chargement et l'instanciation

: Le conteneur charge la classe .class dans la mémoire Java (JVM) et instancie la servlet

initialisation

: La méthode jspInit() est appelée une seule fois par le conteneur après l'instanciation de l'objet.

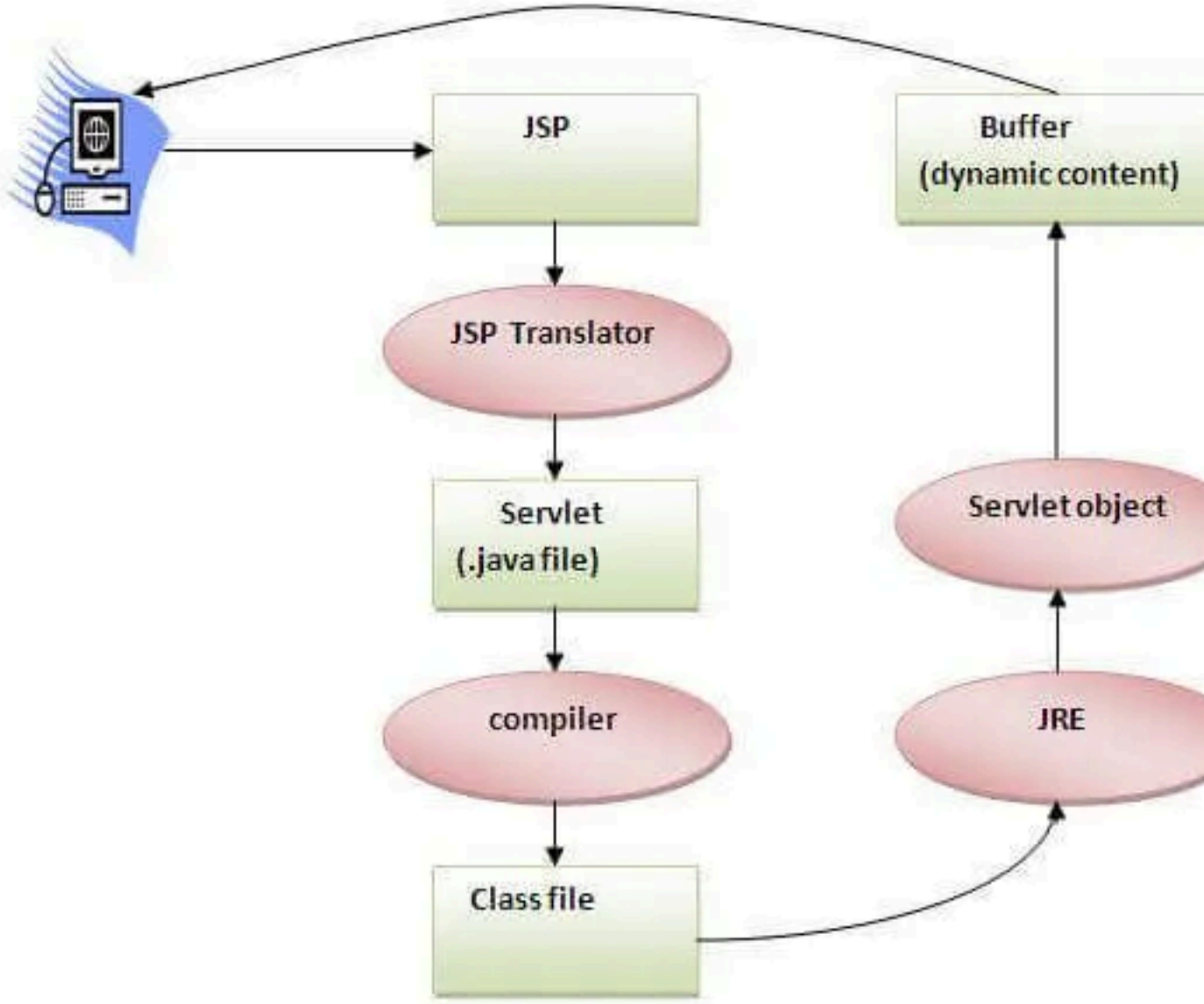
traitement des requêtes : _jspService(request, response)

: méthode appelée à chaque requête

destruction

: à l'arrêt de l'application web ou lorsque que le conteneur décide de décharger la JSP, la méthode jspDestroy() est appelée.

Figure 2.7. Cycle de vie JSP.[1]



Les pages JSP reposent sur :

- les données statiques comme le HTML,
- les directives,
- les scripts et variables,
- les actions,
- les balises personnalisées

3.1. Les bases

Vous trouvez ici une description complète des JSP : `https://www.jmdoudoux.fr/java/dej/chap-jsp.htm#jsp-1`.

[1] source : `https://medium.com/@SachinPandeyOnline/advanced-java-jsp-lifecycle-cd059d1f576b`.

3.1.1. HelloWorld

Déployer et tester le code suivant :

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Hello World JSP</title>
</head>
<%! String text = "Hello World!"; %>
<body>
<h1>
    <%
        out.println(text);
    %>
</h1>
<!-- Alternative plus courte via une expression JSP -->
<p><%= text %></p>
</body>
</html>
```

A quoi correspond <%@ ... %>, <%@ page ... %>, <%! %>, <% ... %>, <%= %> et qu'est ce que out ?

3.1.2. Afficher un tableau de produit

Créer une page JSP qui affiche une ArrayList comprenant les produits suivants, vous réutilisez ici la classe Product crée précédement :

```
List<Product> products = new ArrayList<>();

        products.add(new Product(1, "Ordinateur", 750.0, 10));
        products.add(new Product(2, "Clavier", 35.0, 25));
        products.add(new Product(3, "Souris", 20.0, 50));
        products.add(new Product(4, "Écran", 180.0, 8));
```

3.1.3. Afficher un tableau de produits depuis un dao via une servlet

Dans la version précédente, nous n'avons pas respecté l'approche MVC, les données, l'action est la vue ne sont pas séparées. Nous allons reprendre le dao en mémoire des questions précédantes comme modèle, une servlet comme contrôleur qui va répondre au verbe HTTP get et une JSP comme affichage.

Pour passer l'information de la servlet vers la page JSP vous devrez enrichir la requête avant de la "forwardé" à la page JSP :

```
request.setAttribute("products", products);

request.getRequestDispatcher("/products.jsp")
               .forward(request, response);
```

Dans la JSP vous pourrez récupérer dans la requête les produits :

```
List<Product> products =
        (List<Product>) request.getAttribute("products");
```

Le DAO utilisé sera en mémoire et sera peuplé dans la méthode init de la servlet.

3.1.4. Augmenter et diminuer la quantité de produits

Modifier le code précédent pour pourvoir incrémenter et décrémenter la quantité. L'information sera transmise en Post avec deux formulaires par produit, vous pouvez utiliser des champs hidden pour l'identifiant et l'action souhaitée. Une fois le DAO sollicité, vous devrez faire une redirection en get :

```
response.sendRedirect(
            request.getContextPath() + "/chemin_vers/products_controller";
```

3.1.5. Login

Reprendre avec une approche MVC, l'exemple du formulaire de login de la partie servlet avec cookie et session.

4. Java Persistence API

Nous allons utiliser Hibernate est un framework open source pour Java qui implémente la spécification JPA (Jakarta Persistence API) et sert d'ORM (Object-Relational Mapping). Vous pouvez trouver ici la documentation officielle : `https://docs.hibernate.org/orm/3.5/reference/fr/pdf/hibernate_reference.pdf`. Deux approches sont possibles soit via des sessions, soit via des gestionnaires d'entité (EntityManager), nous allons utiliser cette seconde approche car l'EntityManager est l'interface centrale de la spécification JPA (Jakarta Persistence API), il est une enveloppe standardisée autour de la Session d'Hibernate.

4.1. La structure du projet

Nous allons créer un projet JDK classique, mais en ajoutant un plugin pour hibernate :

```
plugins {
    id 'java'
    id("org.hibernate.orm") version "8.0.0.Beta2"
}
```

Pour les tests nous utiliserons JUnit et Mockito et comme moteur de base de données h2 :

```
dependencies {
    implementation 'com.h2database:h2:2.4.240'

    testImplementation platform('org.junit:junit-bom:5.11.4')
    testImplementation 'org.junit.jupiter:junit-jupiter'

    testImplementation 'org.mockito:mockito-core:5.15.2'
    testImplementation 'org.mockito:mockito-junit-jupiter:5.15.2'

    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'

}
```

4.2. Une première découverte

Il est possible de travailler avec des sessions ou avec un gestionnaire d'entité. Nous favoriserons la seconde approche plus proche de celle choisie par spring boot.

4.2.1. L'entité produit

Nous allons utilise l'entité suivante :

```
package iut.nantes.hibernate.entities;
import java.util.Objects;
import jakarta.persistence.*;

@Entity
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(name = "product_name", nullable = false, length = 150,unique=true)
    private String name;
    @Column(name = "product_price", nullable = false)
    private Double price;
    @Column(name = "product_quantity", nullable = false)
    private Integer quantity;

    public Product() {
```

```
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setPrice(Double price) {
        this.price = price;
    }

    public void setQuantity(Integer quantity) {
        this.quantity = quantity;
    }

    public Product(String name, Double price, Integer quantity) {
        if (price <0 ||  quantity < 0) {
            throw new IllegalArgumentException("price and/or quantity cannot be negative");
        }
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }

    public Integer getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public Double getPrice() {
        return price;
    }

    public Integer getQuantity() {
        return quantity;
    }

    @Override
    public String toString() {
        return "Product{" +
                "id=" + id +
                ", name='" + name + '\'' +
                ", price=" + price +
                ", quantity=" + quantity +
                '}';
    }

    @Override
    public boolean equals(Object o) {
        if (o == null || getClass() != o.getClass()) return false;
        Product product = (Product) o;
        return Objects.equals(id, product.id) && Objects.equals(name, product.name) && Objects.equals(pri
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, name, price, quantity);
    }
}
```

A quoi servent les annotation, qu'est ce qu'un "JavaBean" ?.

4.2.2. Utilisation d'une session

Pour utiliser des sessions, il nous faut définir dans ressources, un fichier `hibernate.cfg.xml` :

```
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD//EN"
        "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>

    <session-factory>
        <property name="connection.url">jdbc:h2:~/test</property>
        <property name="connection.driver_class">org.h2.Driver</property>
        <property name="hibernate.dialect">org.hibernate.dialect.H2Dialect </property>
        <property name="connection.username">sa</property>
        <property name="connection.password"></property>
        <property name="show_sql">true</property>
        <property name="format_sql">true</property>

        <!-- DB schema will be updated if needed -->
        <property name="hibernate.hbm2ddl.auto">create</property>
        <mapping class="iut.nantes.hibernate.entities.Product"/>
    </session-factory>
</hibernate-configuration>
```

Quelle sera l'entité gérée ? Où seront sauvées nos données ?

Tester le main suivant :

```
package iut.nantes.hibernate;
import iut.nantes.hibernate.entities.Product;
import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;
import jakarta.persistence.TypedQuery;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

import java.io.IOException;
import java.util.List;

public class MainSession {
    public static void main(String[] args) throws IOException {

//Avec des sessions hibernate.cfg.xml
        Configuration config = new Configuration();
        config.addAnnotatedClass(iut.nantes.hibernate.entities.Product.class);
        config.configure();




        // local SessionFactory bean created
        SessionFactory sessionFactory = config.buildSessionFactory();
        Session session = sessionFactory.openSession();

        try {
            session.beginTransaction();

            Product product1 = new Product("Test1",10.0,2);
            Product product2 = new Product("Test2",220.0,20);
            session.persist(product1);
            session.persist(product2);
            session.getTransaction().commit();
            List<Product> products;
            products= session.createQuery("SELECT p
              FROM Product p", Product.class).getResultList();
            System.out.println(products);
//TODO Supprimer les produits créés et afficher les produits

        } finally{
            session.close();
            sessionFactory.close();
```

```
        }
    }
}
```

Supprimer product1 et product2 puis afficher les produits.

4.2.3. Utilisation d'un gestionnaire d'entités

Cette fois-ci, nous allons devoir créer dans ressources, un répertoire META-INF qui contient le fichier persistence.xml :

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
             version="3.0">

    <persistence-unit name="myPersistenceUnit">
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

        <class>iut.nantes.hibernate.entities.Product</class>
        <properties>
            <property name="jakarta.persistence.jdbc.driver"
                      value="org.h2.Driver"/>
            <property name="hibernate.dialect" value="org.hibernate.dialect.H2Dialect"/>
            <!-- Travail sur disque
            <property name="jakarta.persistence.jdbc.url"
                      value="jdbc:h2:~/test"/>
                      -->
            <property name="jakarta.persistence.jdbc.url"
                      value="jdbc:h2:mem:test"/>
            <property name="jakarta.persistence.jdbc.user"
                      value="sa"/>
            <property name="jakarta.persistence.jdbc.password"
                      value=""/>

            <property name="hibernate.hbm2ddl.auto"
                      value="update"/>
            <property name="hibernate.show_sql"
                      value="true"/>
            <property name="show_sql" value="true" />
            <property name="format_sql" value="true" />
        </properties>

    </persistence-unit>
</persistence>
```

Quelle est le nom de l'unité de persistance et où sont sauvegardées les données ?

Tester le main suivant :

```
package iut.nantes.hibernate;
import iut.nantes.hibernate.entities.Product;
import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;
import jakarta.persistence.TypedQuery;

import java.io.IOException;
import java.util.List;

public class MainEntityManager {
    public static void main(String[] args) throws IOException {
        EntityManagerFactory emf =
                Persistence.createEntityManagerFactory("myPersistenceUnit");
        EntityManager em = emf.createEntityManager();

        try {
            em.getTransaction().begin();

            // Ne pas fournir d'ID : Hibernate le génère automatiquement
```

```
            Product product1 = new Product("Test1",10.0,2);
            Product product2 = new Product("Test2",220.0,20);
            //Persistance
            em.persist(product1);
            em.persist(product2);

            List<Product> products;
            //Requête
            TypedQuery<Product> query = em.createQuery("from Product", Product.class);
            products= query.getResultList();
            System.out.println(products);

            //Modification
            product1.setPrice(1000.5);
            em.persist(product1);
            products= query.getResultList();
            System.out.println(products);

            //Suppression
            em.remove(product1);
            products= query.getResultList();
            System.out.println(products);

            em.getTransaction().commit();
        } catch (Exception e) {
            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback();
            }
            e.printStackTrace();
        } finally {
            em.close();
            emf.close();
        }

    }
}
```

En utilisant createStatement (`https://javadoc.io/doc/jakarta.persistence/jakarta.persistence-api/latest/jakarta.persistence/jakarta/persistence/EntityManager.html`), supprimer tous les produits puis les afficher.

4.3. Un Dépôt simple

Nous allons créer un dépôt pour les produits en implémentant l'interface CrudRepository dans la classe ProductRepository. Les tests ProductRepositoryTest sont fournis.

Vous pouvez si vous le souhaitez utiliser des transactions pour les opérations en écriture :

```
EntityTransaction transaction = entityManager.getTransaction();

        try {
            transaction.begin();
            entityManager.createStatement("DELETE FROM Product").execute();
            transaction.commit();

        } catch (Exception e) {
            if (transaction.isActive()) {
                transaction.rollback();
            }
            throw e;
        }
```

4.4. Associations OneToMany

Dans le paquetage iut.nantes.hibernate.entities.onetomany créer une entité Product et une Entité User de telle sorte d'un User (OneToMany) puisse avoir plusieurs produits mais qu'un produit ne puisse avoir au plus qu'un seul user (@ManyToOne). Deux entités ne pouvant avoir le même nom vous pouvez le redéfinir comme suit :

```
@Entity(name = "iut.nantes.hibernate.entities.onetomany.Product")
```

Le main suivant doit fonctionner :

```
package iut.nantes.hibernate;

import iut.nantes.hibernate.entities.onetomany.Product;
import iut.nantes.hibernate.entities.onetomany.User;

import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;

public class Main {

    public static void main(String[] args) {

        EntityManagerFactory emf =
                Persistence.createEntityManagerFactory("myPersistenceUnit");

        EntityManager em = emf.createEntityManager();

        try {
            // 1. Création d'un utilisateur
            User user = new User("amine");

            // 2. Création des produits
            Product p1 = new Product("PC", 1200.0, 5);
            Product p2 = new Product("Clavier", 50.0, 10);
            Product p3 = new Product("Souris", 25.0, 15);

            // 3. Association des produits à l'utilisateur
            user.addProduct(p1);
            user.addProduct(p2);
            user.addProduct(p3);

            // 4. Enregistrement en base de données
            em.getTransaction().begin();

            em.persist(user);

            em.getTransaction().commit();

            System.out.println("Utilisateur et produits enregistrés !");

            // 5. Récupération de l'utilisateur
            em.clear();

            User foundUser = em.find(User.class, user.getId());

            System.out.println("\nUtilisateur : " + foundUser.getUsername());

            // 6. Affichage des produits
            System.out.println("Liste des produits :");

            for (Product product : foundUser.getProducts()) {
                System.out.println(product);
            }

            // 7. Suppression d'un produit
            em.getTransaction().begin();

            Product productToRemove = foundUser.getProducts().get(0);
            foundUser.removeProduct(productToRemove);

            em.getTransaction().commit();

            System.out.println("\nProduit supprimé !");
```

```
            // 8. Vérification
            System.out.println("Produits restants :");

            for (Product product : foundUser.getProducts()) {
                System.out.println(product);
            }

        } catch (Exception e) {

            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback();
            }

            e.printStackTrace();

        } finally {
            em.close();
            emf.close();
        }
    }
}
```

4.5. Associations ManyToMany

Cette fois ci nous souhaitons qu'un utilisateur puisse posséder plusieurs produit et qu'un produit puisse être possédé par plusieurs utilisateurs. Le main suivant doit fonctionner.

```
package iut.nantes.hibernate.entities.manytomany;

import iut.nantes.hibernate.entities.manytomany.Product;
import iut.nantes.hibernate.entities.manytomany.User;

import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;

public class Main {

    public static void main(String[] args) {

        EntityManagerFactory emf =
                Persistence.createEntityManagerFactory("myPersistenceUnit");

        EntityManager em = emf.createEntityManager();

        try {
            em.getTransaction().begin();

            // Création des utilisateurs
            User user1 = new User("Alice");
            User user2 = new User("Bob");

            // Création des produits
            Product product1 = new Product("Ordinateur", 899.99, 10);
            Product product2 = new Product("Clavier", 49.99, 25);
            Product product3 = new Product("Souris", 19.99, 50);

            // Association ManyToMany
            product1.addUser(user1);
            product1.addUser(user2);

            product2.addUser(user1);

            product3.addUser(user2);

            // Persistance
            em.persist(user1);
            em.persist(user2);
```

```
            em.persist(product1);
            em.persist(product2);
            em.persist(product3);

            em.getTransaction().commit();

            // Affichage des associations
            System.out.println("Produits de Alice :");

            User alice = em.find(User.class, user1.getId());

            for (Product product : alice.getProducts()) {
                System.out.println(
                        product.getName() + " - " + product.getPrice() + " €"
                );
            }

            System.out.println("\nUtilisateurs du produit Ordinateur :");

            Product ordinateur = em.find(Product.class, product1.getId());

            for (User user : ordinateur.getUsers()) {
                System.out.println(user.getUsername());
            }

        } catch (Exception e) {
            if (em.getTransaction().isActive()) {
                em.getTransaction().rollback();
            }
            e.printStackTrace();
        } finally {
            em.close();
            emf.close();
        }
    }
}
```

4.6. ManyToMany avec des paramètres

Nous souhaitons maintenant qu'un User possède des produits avec une quantité choisie, un qu'un produit puisse être possédé par plusieurs users. Il existe deux approche, la plus "élégante" d'un point de vue objet est d'avoir une classe imbriquée UserProdut dans la classe Produit :

```
// Nested class
    @Entity
    @Table(name = "user_product")
    public static class UserProduct {

        @Id
        @GeneratedValue(strategy = GenerationType.IDENTITY)
        private Integer id;

        @ManyToOne
        @JoinColumn(name = "user_id")
        private User user;

        @ManyToOne
        @JoinColumn(name = "product_id")
        private Product product;

        private Integer quantity;

        public UserProduct() {}

        public UserProduct(User user, Product product, Integer quantity) {
            this.user = user;
            this.product = product;
            this.quantity = quantity;
        }
```

```
    }
```

La recommandation d'hibernate est de passer par une entité externe UserProduct :

```
package iut.nantes.hibernate.entities.manytomanywithparameter;

import jakarta.persistence.*;

@Entity(name="iut.nantes.hibernate.entities.manytomanywithparameter.UserProduct")
@Table(name = "user_product")
public class UserProduct {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @ManyToOne
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @ManyToOne
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;

    @Column(nullable = false)
    private Integer quantity;

    public UserProduct() {}

    public UserProduct(User user, Product product, Integer quantity) {
        if (quantity < 0) {
            throw new IllegalArgumentException(
                    "Quantity cannot be negative"
            );
        }

        this.user = user;
        this.product = product;
        this.quantity = quantity;
    }

    public User getUser() {
        return user;
    }

    public Product getProduct() {
        return product;
    }

    public Integer getQuantity() {
        return quantity;
    }

    public void setQuantity(Integer quantity) {
        if (quantity < 0) {
            throw new IllegalArgumentException(
                    "Quantity cannot be negative"
            );
        }
        this.quantity = quantity;
    }
}
```

Coder User et Product qui seront en OneToMany avec UserProduct. Créer un main pour tester.

Chapitre 3. Une première application MVC

Nous sommes suffisamment robuste pour commencer la programmation coté serveur.

//TODO

Chapitre 4. Utilisation du framework MVC spring boot